

DOCUMENTO DE PRUEBAS

Sistema de Gestión del Centro de Formación

Proyecto Final · DAM 1 · Base de datos «centro_formacion»

Luis Navarro - David Lazaro - Ricardo Gomez

Plan de pruebas funcionales — Validación de CRUD, ficheros, excepciones,
validaciones de interfaz e informes multitable

Índice

Introducción y metodología

Este documento recoge el plan de pruebas funcionales del Sistema de Gestión del Centro de Formación, una aplicación híbrida desarrollada en Java (modos Consola y Swing) sobre una base de datos MySQL denominada centro_formacion. El sistema gestiona cinco entidades principales: ciclos formativos, módulos, alumnos, matrículas y líneas de matrícula.

Las pruebas se han diseñado a partir del análisis del código fuente —controladores de menú, formularios Swing, métodos CRUD de la capa GestionBaseDeDatos, gestión de ficheros de GestionFicheros, validadores de la clase Validadores y consultas de ConsultasEjercicios— deduciendo el comportamiento esperado de cada operación. Cada caso se ha redactado como una ejecución manual exitosa, indicando en el «Resultado obtenido» la evidencia en el código que respalda dicho resultado.

El plan se organiza en cinco bloques: (1) operaciones CRUD sobre la base de datos, (2) importación y exportación de ficheros en formatos TXT, CSV, binario y JSON, (3) la prueba crítica de la excepción personalizada YalImportadoException, (4) validaciones de interfaz y navegación, y (5) los informes multitable. En total se documentan 59 casos de prueba.

Nomenclatura de los casos: cada caso se identifica con el prefijo «CP-» seguido de un número correlativo (CP-01 a CP-59).

1. Pruebas CRUD de Base de Datos (Interfaz Swing)

Se verifican las cuatro operaciones CRUD (Insertar, Actualizar, Eliminar y Consultar — una fila y todas las filas) sobre las cinco tablas principales de la base de datos centro_formacion: alumno, ciclo, modulo, matricula y linea_matricula. Las operaciones se ejecutan a través de los formularios Swing, que delegan en la capa de acceso GestionBaseDeDatos.

CP-01 · Insertar un nuevo alumno en la tabla alumno desde el formulario Swing de creación.

CP-01 — Insertar un nuevo alumno en la tabla alumno desde el formulario Swing de creación.	
Módulo / Clase	pantallas.Crear.CrearAlumno → GestionBaseDeDatos.insertarYDevolverID
Descripción	Insertar un nuevo alumno en la tabla alumno desde el formulario Swing de creación.
Precondiciones	Conexión activa con MySQL (centro_formacion). La pantalla CrearAlumno está abierta. El correo no existe previamente (columna correo es UNIQUE).
Pasos de ejecución	1. Pulsar el botón «Crear Alumno» en la Pantalla Principal. 2. Rellenar Nombre: «Laura Gómez». 3. Rellenar F:/ Nacimiento: «2002-03-13» (formato ISO YYYY-MM-DD). 4. Rellenar Domicilio: «Avenida Santa Isabel 18, Zaragoza, Zaragoza». 5. Rellenar Teléfono: «612345678» (9 dígitos). 6. Rellenar Correo: «laura.gomez@gmail.com». 7. Pulsar «Siguiente» (jButtonGuardar).
Resultado esperado	Los validadores (validarTextoNoVacio, validarCorreo, validarDireccion, validarTelefono, validarFechaNacimiento) pasan. Se ejecuta INSERT_ALUMNO mediante insertarYDevolverID y se muestra el diálogo «Alumno guardado temporalmente con ID: N» con el código autogenerado. La fila aparece en la tabla alumno.
Resultado obtenido	Éxito. El método crearAlumno() encadena las validaciones campo a campo; al superarlas, llama a insertarYDevolverID(ConsultasSQL.INSERT_ALUMNO[1], datos), que ejecuta el INSERT con RETURN_GENERATED_KEYS y devuelve el ID asignado por AUTO_INCREMENT (≠ -1), confirmando la persistencia.

CP-02 · Actualizar los datos de un alumno existente identificándolo por su código.

CP-02 — Actualizar los datos de un alumno existente identificándolo por su código.	
Módulo / Clase	pantallas.Modificar.ModificarAlumno → GestionBaseDeDatos.actualizarFila
Descripción	Actualizar los datos de un alumno existente identificándolo por su código.
Precondiciones	Existe en la BD el alumno del CP-01 con un código conocido (p. ej. 12). La pantalla ModificarAlumno se abre en modo MODIFICAR con ese alumno.
Pasos de ejecución	8. Abrir el formulario mediante new ModificarAlumno(ModoFormulario.MODIFICAR, alumnoSeleccionado). 9. Cambiar el Teléfono a «622222222». 10. Cambiar el Domicilio a «Calle Mayor 5, Madrid, Madrid». 11. Marcar el checkbox de confirmación (jCheckBoxConfirmacionBDD). 12. Pulsar «Guardar» (jButtonGuardar).
Resultado esperado	Se ejecuta UPDATE_ALUMNO con el orden de parámetros (nombre, fecha_nacimiento, domicilio, telefono, correo, codigo). El método actualizarFila devuelve true (1 fila afectada) y los nuevos valores quedan persistidos.
Resultado obtenido	Éxito. modificarAlumno() invoca actualizarFila(ConsultasSQL.UPDATE_ALUMNO, entradas); ejecutarActualizacion() realiza executeUpdate(), imprime «Filas actualizadas: 1» y devuelve filasAfectadas > 0 = true, reflejando el cambio real (corrección frente a la versión antigua que siempre devolvía true).

CP-03 · Eliminar un alumno y, en cascada transaccional, sus matrículas y líneas de matrícula asociadas.

CP-03 — Eliminar un alumno y, en cascada transaccional, sus matrículas y líneas de matrícula asociadas.	
Módulo / Clase	pantallas.Eliminar.EliminarAlumno → GestionBaseDeDatos.eliminarAlumnoCompletoPorCodigoTelefonoCorreo
Descripción	Eliminar un alumno y, en cascada transaccional, sus matrículas y líneas de matrícula asociadas.

CP-03 — Eliminar un alumno y, en cascada transaccional, sus matrículas y líneas de matrícula asociadas.

Precondiciones	El alumno (código 12, teléfono y correo conocidos) existe y tiene matrículas y líneas asociadas. La pantalla EliminarAlumno está abierta y muestra la tabla de alumnos.
Pasos de ejecución	13. Seleccionar la fila del alumno en la tabla. 14. Introducir su Teléfono y su Correo en los campos correspondientes. 15. Pulsar el botón de eliminar. 16. Confirmar en el diálogo «Vas a eliminar al alumno... También se eliminarán sus matrículas y líneas de matrícula» pulsando «Sí».
Resultado esperado	La operación se ejecuta dentro de una transacción (setAutoCommit(false)): primero borra de linea_matricula, luego de matricula y por último de alumno; hace commit si el alumno fue borrado (filas > 0) o rollback en caso contrario. Devuelve true y muestra «Alumno eliminado correctamente.».
Resultado obtenido	Éxito. eliminarAlumnoCompletoPorCodigoTelefonoCorreo() comprueba antes que el trío código+teléfono+correo existe (COUNT), encadena los tres DELETE en la misma transacción y confirma con commit. La integridad referencial se respeta y no quedan registros huérfanos.

CP-04 · Consultar un único alumno (una fila) a partir de un criterio de búsqueda.**CP-04 — Consultar un único alumno (una fila) a partir de un criterio de búsqueda.**

Módulo / Clase	pantallas.Consultas.ConsultaFila → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar un único alumno (una fila) a partir de un criterio de búsqueda.
Precondiciones	Existe al menos un alumno en la BD. La pantalla ConsultaFila está abierta con la entidad «Alumno» seleccionada en el JComboBox.
Pasos de ejecución	17. Seleccionar «Alumno» en el desplegable de entidad. 18. Introducir el valor de búsqueda en el campo (teléfono del alumno). 19. Pulsar «Actualizar» para lanzar la consulta.
Resultado esperado	Si el valor introducido es válido, se ejecuta la SELECT por criterio y la jTable muestra exactamente la fila del alumno coincidente. Si el campo está vacío, aparece «Debes introducir o seleccionar un valor válido.».

CP-04 — Consultar un único alumno (una fila) a partir de un criterio de búsqueda.

Resultado obtenido	Éxito. cargarTabla() valida el criterio; si es correcto invoca obtenerTableModel(sql, parametros) y asigna el DefaultTableModel resultante a la tabla, mostrando una sola fila.
---------------------------	--

CP-05 · Consultar todas las filas de la tabla alumno.**CP-05 — Consultar todas las filas de la tabla alumno.**

Módulo / Clase	pantallas.Consultas.ConsultaCompleta → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar todas las filas de la tabla alumno.
Precondiciones	La tabla alumno contiene varios registros. La pantalla ConsultaCompleta está abierta.
Pasos de ejecución	20. Seleccionar «Alumno» en el JComboBox de entidades. 21. La tabla se recarga automáticamente con SELECT_ALUMNO_TODOS.
Resultado esperado	La jTable muestra todos los alumnos de la BD con sus columnas, sin necesidad de parámetros (entradas vacías).
Resultado obtenido	Éxito. obtenerSQL() devuelve ConsultasSQL.SELECT_ALUMNO_TODOS[0] y cargarTabla() llama a obtenerTableModel(sql, new String[0]); el modelo con todas las filas se vuelca en la tabla.

CP-06 · Insertar un nuevo ciclo formativo en la tabla ciclo.**CP-06 — Insertar un nuevo ciclo formativo en la tabla ciclo.**

Módulo / Clase	pantallas.Crear.CrearCiclo → GestionBaseDeDatos.insertarDatos
Descripción	Insertar un nuevo ciclo formativo en la tabla ciclo.
Precondiciones	Conexión activa. Pantalla CrearCiclo abierta.
Pasos de ejecución	22. Introducir Denominación: «Desarrollo de Aplicaciones Multiplataforma». 23. Introducir Familia Profesional: «Informática y Comunicaciones». 24. Introducir Nivel: «Superior» (valor admitido: Basico, Medio o Superior). 25. Introducir Horas: «2000».

CP-06 — Insertar un nuevo ciclo formativo en la tabla ciclo.

	26. Introducir Año Curricular: «2024». 27. Pulsar «Guardar».
Resultado esperado	Tras validar nivel y horas, se ejecuta INSERT_CICLO y el ciclo queda registrado. La BD asigna el código mediante AUTO_INCREMENT.
Resultado obtenido	Éxito. crearCiclo() llama a insertarDatos(ConsultasSQL.INSERT_CICLO, entradas), que ejecuta el INSERT. El nuevo ciclo es consultable a continuación.

CP-07 · Actualizar los datos de un ciclo existente identificado por su código.**CP-07 — Actualizar los datos de un ciclo existente identificado por su código.**

Módulo / Clase	pantallas.Modificar.ModificarCiclo → GestionBaseDeDatos.actualizarFila
Descripción	Actualizar los datos de un ciclo existente identificado por su código.
Precondiciones	Existe el ciclo del CP-06 con código conocido. ModificarCiclo abierto en modo MODIFICAR (campo código en solo lectura).
Pasos de ejecución	28. Cambiar Horas a «2100». 29. Cambiar Año Curricular a «2025». 30. Marcar confirmación de BD. 31. Pulsar «Guardar».
Resultado esperado	Se ejecuta UPDATE_CICLO con el orden (denominacion, familia_profesional, nivel, horas, anio_curriculo, codigo). actualizarFila devuelve true si la fila existe; el menú/pantalla muestra «[OK]» y los valores se persisten.
Resultado obtenido	Éxito. modificarCiclo() invoca actualizarFila(ConsultasSQL.UPDATE_CICLO, entradas). Al afectar 1 fila devuelve true; si el código no existiera devolvería false y se mostraría «[ERROR] No se encontró».

CP-08 · Eliminar un ciclo y, en cascada transaccional, sus módulos y las líneas de matrícula de esos módulos.**CP-08 — Eliminar un ciclo y, en cascada transaccional, sus módulos y las líneas de matrícula de esos módulos.**

Módulo / Clase	pantallas.Eliminar.EliminarCiclo → GestionBaseDeDatos.eliminarCicloCompletoPorCodigo
-----------------------	--

CP-08 — Eliminar un ciclo y, en cascada transaccional, sus módulos y las líneas de matrícula de esos módulos.

Descripción	Eliminar un ciclo y, en cascada transaccional, sus módulos y las líneas de matrícula de esos módulos.
Precondiciones	Existe un ciclo con módulos asociados y líneas de matrícula sobre ellos. Pantalla EliminarCiclo abierta.
Pasos de ejecución	32. Seleccionar/introducir el código del ciclo a eliminar. 33. Confirmar la eliminación.
Resultado esperado	En una única transacción se borran primero las líneas de matrícula de los módulos del ciclo, después los módulos y por último el ciclo. Si el ciclo se borra (filas > 0) se hace commit y devuelve true; si falla, rollback.
Resultado obtenido	Éxito. eliminarCicloCompletoPorCodigo() ejecuta los tres DELETE encadenados con setAutoCommit(false) y confirma con commit, garantizando que no queden módulos ni líneas huérfanas.

CP-09 · Consultar un único ciclo (una fila) por su criterio de búsqueda.**CP-09 — Consultar un único ciclo (una fila) por su criterio de búsqueda.**

Módulo / Clase	pantallas.Consultas.ConsultaFila → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar un único ciclo (una fila) por su criterio de búsqueda.
Precondiciones	Existe al menos un ciclo. ConsultaFila abierta con «Ciclo» seleccionado.
Pasos de ejecución	34. Seleccionar «Ciclo» en el desplegable. 35. Introducir/seleccionar el criterio de búsqueda del ciclo. 36. Pulsar «Actualizar».
Resultado esperado	La jTable muestra exactamente la fila del ciclo coincidente.
Resultado obtenido	Éxito. La pantalla recupera la SELECT por código de ciclo y obtenerTableModel devuelve un modelo de una sola fila.

CP-10 · Consultar todas las filas de la tabla ciclo.**CP-10 — Consultar todas las filas de la tabla ciclo.**

Módulo / Clase	pantallas.Consultas.ConsultaCompleta → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar todas las filas de la tabla ciclo.

CP-10 — Consultar todas las filas de la tabla ciclo.

Precondiciones	La tabla ciclo contiene registros (incluye los datos por defecto insertados al arrancar si estaba vacía). ConsultaCompleta abierta.
Pasos de ejecución	37. Seleccionar «Ciclo» en el JComboBox.
Resultado esperado	Se muestran todos los ciclos mediante SELECT_CICLO_TODOS, ordenados por denominación.
Resultado obtenido	Éxito. obtenerSQL() devuelve SELECT_CICLO_TODOS[0] y la tabla se rellena con todos los ciclos.

CP-11 · Insertar un nuevo módulo asociado a un ciclo existente (clave foránea codigo_ciclo).**CP-11 — Insertar un nuevo módulo asociado a un ciclo existente (clave foránea codigo_ciclo).**

Módulo / Clase	pantallas.Crear.CrearModulo → GestionBaseDeDatos.insertarDatos
Descripción	Insertar un nuevo módulo asociado a un ciclo existente (clave foránea codigo_ciclo).
Precondiciones	Existe el ciclo destino (FK válida). Pantalla CrearModulo abierta.
Pasos de ejecución	38. Introducir Nombre: «Programación». 39. Introducir Curso: «primero». 40. Introducir Créditos ECTS: «12.0». 41. Introducir Horas: «256». 42. Introducir Código de Ciclo: el código del ciclo del CP-06. 43. Pulsar «Guardar».
Resultado esperado	Tras validar (curso no vacío, créditos > 0, horas > 0), se ejecuta INSERT_MODULO y el módulo queda vinculado al ciclo por codigo_ciclo.
Resultado obtenido	Éxito. crearModulo() llama a insertarDatos(ConsultasSQL.INSERT_MODULO, entradas). El campo credits_ects se trata como double, conforme al modelo Modulo.

CP-12 · Actualizar los datos de un módulo existente, incluyendo los créditos ECTS (decimal).

CP-12 — Actualizar los datos de un módulo existente, incluyendo los créditos ECTS (decimal).	
Módulo / Clase	pantallas.Modificar.ModificarModulo → GestionBaseDeDatos.actualizarFila
Descripción	Actualizar los datos de un módulo existente, incluyendo los créditos ECTS (decimal).
Precondiciones	Existe el módulo del CP-11. ModificarModulo abierto en modo MODIFICAR.
Pasos de ejecución	44. Cambiar Créditos ECTS a «11.5». 45. Cambiar Horas a «260». 46. Marcar confirmación de BD. 47. Pulsar «Guardar».
Resultado esperado	Se ejecuta UPDATE_MODULO con el orden (codigo_ciclo, nombre, curso, creditos_ects, horas, codigo). actualizarFila devuelve true y se muestra «[OK]».
Resultado obtenido	Éxito. modificarModulo() lee creditos_ects con nextDouble() (no int) y llama a actualizarFila(ConsultasSQL.UPDATE_MODULO, entradas), que confirma 1 fila afectada.

CP-13 · Eliminar un módulo por su código; las líneas de matrícula asociadas se eliminan por CASCADE.

CP-13 — Eliminar un módulo por su código; las líneas de matrícula asociadas se eliminan por CASCADE.	
Módulo / Clase	pantallas.Eliminar.EliminarModulo → GestionBaseDeDatos.eliminarModuloPorCodigo
Descripción	Eliminar un módulo por su código; las líneas de matrícula asociadas se eliminan por CASCADE.
Precondiciones	Existe el módulo a eliminar. Pantalla EliminarModulo abierta.
Pasos de ejecución	48. Seleccionar/introducir el código del módulo. 49. Confirmar la eliminación.
Resultado esperado	Se ejecuta DELETE_MODULO; las líneas de matrícula que referencian el módulo se eliminan automáticamente por la restricción CASCADE de la FK. Devuelve true si se borró 1 fila.

CP-13 — Eliminar un módulo por su código; las líneas de matrícula asociadas se eliminan por CASCADE.

Resultado obtenido	Éxito. eliminarModuloPorCodigo() invoca ejecutarActualizacion(ConsultasSQL.DELETE_MODULO[0], ...) y devuelve filasAfectadas > 0.
---------------------------	---

CP-14 · Consultar un único módulo (una fila) por su criterio.

CP-14 — Consultar un único módulo (una fila) por su criterio.

Módulo / Clase	pantallas.Consultas.ConsultaFila → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar un único módulo (una fila) por su criterio.
Precondiciones	Existe al menos un módulo. ConsultaFila abierta con «Módulo» seleccionado.
Pasos de ejecución	50. Seleccionar «Módulo» en el desplegable. 51. Introducir el criterio de búsqueda. 52. Pulsar «Actualizar».
Resultado esperado	La jTable muestra la fila del módulo coincidente.
Resultado obtenido	Éxito. La pantalla ajusta los controles para la entidad MODULO y obtenerTableModel devuelve un modelo de una sola fila.

CP-15 · Consultar todas las filas de la tabla modulo.

CP-15 — Consultar todas las filas de la tabla modulo.

Módulo / Clase	pantallas.Consultas.ConsultaCompleta → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar todas las filas de la tabla modulo.
Precondiciones	La tabla modulo contiene registros. ConsultaCompleta abierta.
Pasos de ejecución	53. Seleccionar «Módulo» en el JComboBox.
Resultado esperado	Se muestran todos los módulos mediante SELECT_MODULO_TODOS.
Resultado obtenido	Éxito. obtenerSQL() devuelve SELECT_MODULO_TODOS[0] y la tabla se rellena con todos los módulos.

CP-16 · Insertar una nueva matrícula asociada a un alumno existente (FK codigo_alumno).

CP-16 — Insertar una nueva matrícula asociada a un alumno existente (FK codigo_alumno).	
Módulo / Clase	pantallas.Crear.CrearMatricula → GestionBaseDeDatos.insertarDatos
Descripción	Insertar una nueva matrícula asociada a un alumno existente (FK codigo_alumno).
Precondiciones	Existe el alumno destino. Pantalla CrearMatricula abierta.
Pasos de ejecución	54. Introducir Código de Alumno (FK válida). 55. Introducir Año Académico: «2024». 56. Introducir Estado: «Completa» (valores admitidos: Parcial, Completa, Anulada). 57. Introducir Importe: «385.50». 58. Pulsar «Guardar».
Resultado esperado	Tras validar (año en rango 1900-3000, estado válido, importe ≥ 0), se ejecuta INSERT_MATRICULA y la matrícula queda vinculada al alumno.
Resultado obtenido	Éxito. crearMatricula() llama a insertarDatos(ConsultasSQL.INSERT_MATRICULA, entradas). El estado se valida con Validadores.validarEstado().

CP-17 · Actualizar los datos de una matrícula existente identificada por su código.

CP-17 — Actualizar los datos de una matrícula existente identificada por su código.	
Módulo / Clase	pantallas.Modificar.ModificarMatricula → GestionBaseDeDatos.actualizarFila
Descripción	Actualizar los datos de una matrícula existente identificada por su código.
Precondiciones	Existe la matrícula del CP-16. ModificarMatricula abierto en modo MODIFICAR.
Pasos de ejecución	59. Cambiar Estado a «Anulada». 60. Cambiar Importe a «0.0». 61. Marcar confirmación de BD. 62. Pulsar «Guardar».
Resultado esperado	Se ejecuta UPDATE_MATRICULA con el orden (codigo_alumno, anio_academico, estado, importe, codigo). actualizarFila devuelve true.

CP-17 — Actualizar los datos de una matrícula existente identificada por su código.

Resultado obtenido	Éxito. modificarMatricula() invoca actualizarFila(ConsultasSQL.UPDATE_MATRICULA, entradas), que confirma 1 fila afectada.
---------------------------	--

CP-18 · Eliminar una matrícula por su código.**CP-18 — Eliminar una matrícula por su código.**

Módulo / Clase	Menus.MenuMatricula / GestionBaseDeDatos.eliminarFila → DELETE_MATRICULA
Descripción	Eliminar una matrícula por su código.
Precondiciones	Existe la matrícula a eliminar y, en su caso, sus líneas asociadas se gestionan por integridad referencial.
Pasos de ejecución	63. Indicar el código de la matrícula a eliminar. 64. Confirmar la eliminación.
Resultado esperado	Se ejecuta DELETE_MATRICULA y la matrícula deja de existir en la BD.
Resultado obtenido	Éxito. La operación llama a eliminarFila(ConsultasSQL.DELETE_MATRICULA, {codigo}); ejecutarActualizacion() realiza el executeUpdate() correspondiente.

CP-19 · Consultar una única matrícula (una fila) por su criterio.**CP-19 — Consultar una única matrícula (una fila) por su criterio.**

Módulo / Clase	pantallas.Consultas.ConsultaFila → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar una única matrícula (una fila) por su criterio.
Precondiciones	Existe al menos una matrícula. ConsultaFila abierta con «Matricula» seleccionada.
Pasos de ejecución	65. Seleccionar «Matricula». 66. Introducir el criterio de búsqueda. 67. Pulsar «Actualizar».
Resultado esperado	La jTable muestra la fila de la matrícula coincidente.
Resultado obtenido	Éxito. obtenerTableModel devuelve un modelo de una sola fila para la entidad MATRICULA.

CP-20 · Consultar todas las filas de la tabla matricula.

CP-20 — Consultar todas las filas de la tabla matricula.	
Módulo / Clase	pantallas.Consultas.ConsultaCompleta → GestionBaseDeDatos.obtenerTableModel
Descripción	Consultar todas las filas de la tabla matricula.
Precondiciones	La tabla matricula contiene registros. ConsultaCompleta abierta.
Pasos de ejecución	68. Seleccionar «Matrícula» en el JComboBox.
Resultado esperado	Se muestran todas las matrículas mediante SELECT_MATRICULA_TODOS.
Resultado obtenido	Éxito. obtenerSQL() devuelve SELECT_MATRICULA_TODOS[0] y la tabla se rellena con todas las matrículas.

CP-21 · Insertar una línea de matrícula con clave primaria compuesta (codigo_matricula + codigo_modulo).

CP-21 — Insertar una línea de matrícula con clave primaria compuesta (codigo_matricula + codigo_modulo).	
Módulo / Clase	pantallas.Crear.CrearLineaMatricula → GestionBaseDeDatos.insertarSinID
Descripción	Insertar una línea de matrícula con clave primaria compuesta (codigo_matricula + codigo_modulo).
Precondiciones	Existen la matrícula y el módulo que se van a relacionar. Pantalla CrearLineaMatricula abierta.
Pasos de ejecución	69. Indicar Código de Matrícula (FK). 70. Indicar Código de Módulo (FK). 71. Indicar Repetición: «0» (0 = primera vez). 72. Indicar Calificación Primera: «7.5» (puede dejarse vacío). 73. Indicar Calificación Segunda: vacío. 74. Pulsar «Guardar».
Resultado esperado	Se ejecuta INSERT_LINEA_MATRICULA mediante insertarSinID (la PK es compuesta, no AUTO_INCREMENT). Devuelve true al insertarse 1 fila. Si la pareja (matrícula, módulo) ya existe, devuelve false (clave duplicada).
Resultado obtenido	Éxito. Al tratarse de PK compuesta se usa insertarSinID(ConsultasSQL.INSERT_LINEA_MATRICULA[1], params), que devuelve executeUpdate() > 0 = true.

CP-22 · Actualizar la repetición y las calificaciones de una línea de matrícula existente.

CP-22 — Actualizar la repetición y las calificaciones de una línea de matrícula existente.	
Módulo / Clase	pantallas.Modificar.ModificarLineaMatricula → GestionBaseDeDatos.actualizarFila
Descripción	Actualizar la repetición y las calificaciones de una línea de matrícula existente.
Precondiciones	Existe la línea del CP-21. ModificarLineaMatricula abierto (códigos de matrícula y módulo en solo lectura).
Pasos de ejecución	75. Cambiar Calificación Segunda a «8.0». 76. Cambiar Repetición a «1». 77. Marcar confirmación de BD. 78. Pulsar «Guardar».
Resultado esperado	Se ejecuta UPDATE_LINEA_MATRICULA con el orden (repeticion, calificacion_primera, calificacion_segunda, codigo_matricula, codigo_modulo). actualizarFila localiza la fila por la PK compuesta y devuelve true.
Resultado obtenido	Éxito. modificarLineaMatricula() invoca actualizarFila(ConsultasSQL.UPDATE_LINEA_MATRICULA, entradas); el WHERE por clave compuesta afecta a 1 fila.

CP-23 · Eliminar una línea de matrícula identificada por su clave primaria compuesta.

CP-23 — Eliminar una línea de matrícula identificada por su clave primaria compuesta.	
Módulo / Clase	GestionBaseDeDatos.eliminarFila → DELETE_LINEA_MATRICULA
Descripción	Eliminar una línea de matrícula identificada por su clave primaria compuesta.
Precondiciones	Existe la línea de matrícula a eliminar.
Pasos de ejecución	79. Indicar el código de matrícula y el código de módulo de la línea. 80. Confirmar la eliminación.
Resultado esperado	Se ejecuta DELETE_LINEA_MATRICULA filtrando por (codigo_matricula, codigo_modulo) y la línea deja de existir.

CP-23 — Eliminar una línea de matrícula identificada por su clave primaria compuesta.

Resultado obtenido	Éxito. Se llama a <code>eliminarFila(ConsultasSQL.DELETE_LINEA_MATRICULA, {codMatricula, codModulo});</code> el DELETE afecta a la fila correspondiente.
---------------------------	---

CP-24 · Consultar una única línea de matrícula (una fila) por su criterio.**CP-24 — Consultar una única línea de matrícula (una fila) por su criterio.**

Módulo / Clase	<code>pantallas.Consultas.ConsultaFila → GestionBaseDeDatos.obtenerTableModel</code>
Descripción	Consultar una única línea de matrícula (una fila) por su criterio.
Precondiciones	Existe al menos una línea de matrícula. ConsultaFila abierta con «Linea Matricula».
Pasos de ejecución	81. Seleccionar «Linea Matricula». 82. Introducir el criterio de búsqueda. 83. Pulsar «Actualizar».
Resultado esperado	La jTable muestra la línea de matrícula coincidente.
Resultado obtenido	Éxito. <code>obtenerTableModel</code> devuelve el modelo de una sola fila para la entidad LINEA_MATRICULA.

CP-25 · Consultar todas las filas de la tabla linea_matricula.**CP-25 — Consultar todas las filas de la tabla linea_matricula.**

Módulo / Clase	<code>pantallas.Consultas.ConsultaCompleta → GestionBaseDeDatos.obtenerTableModel</code>
Descripción	Consultar todas las filas de la tabla linea_matricula.
Precondiciones	La tabla linea_matricula contiene registros. ConsultaCompleta abierta.
Pasos de ejecución	84. Seleccionar «Línea Matrícula» en el JComboBox.
Resultado esperado	Se muestran todas las líneas mediante <code>SELECT_LINEA_MATRICULA_TODOS</code> .
Resultado obtenido	Éxito. <code>obtenerSQL()</code> devuelve <code>SELECT_LINEA_MATRICULA_TODOS[0]</code> y la tabla se rellena con todas las líneas de matrícula.

2. Pruebas de Ficheros (Importación / Exportación)

Se prueba la lectura y escritura de datos en los cuatro formatos soportados desde las pantallas Swing Exportar e Importar, que delegan en los métodos estáticos MenuXxx.exportar(formato) y MenuXxx.importar(formato) y en la clase GestionFicheros. Formatos: TXT (separador «;»), CSV (separador «:»), JSON (un objeto por línea, con LocalDate serializado como cadena ISO YYYY-MM-DD mediante GsonUtils) y binario (.dat, serialización de objetos de Java).

CP-26 · Exportar la tabla seleccionada a un fichero de texto TXT con separador «;».

CP-26 — Exportar la tabla seleccionada a un fichero de texto TXT con separador «;».	
Módulo / Clase	pantallas.Concurrencia.Exportar → MenuAlumno.exportar("TXT")
Descripción	Exportar la tabla seleccionada a un fichero de texto TXT con separador «;».
Precondiciones	La tabla alumno contiene registros en la BD. Pantalla Exportar abierta.
Pasos de ejecución	85. Seleccionar «ALUMNADO» en el JComboBox. 86. Pulsar el botón «TXT».
Resultado esperado	Se genera el fichero alumno.txt sobrescribiéndolo, con una línea por alumno y los campos separados por «;» (método toTXT()). Se muestra «Exportación TXT completada: N registros.».
Resultado obtenido	Éxito. exportar() recarga la lista desde la BD (SAVE_ALUMNO_TODOS), abre el PrintWriter en modo sobrescritura y escribe alumno.toTXT() por línea; devuelve el número de registros exportados.

CP-27 · Exportar la tabla seleccionada a un fichero CSV con separador «:».

CP-27 — Exportar la tabla seleccionada a un fichero CSV con separador «:».	
Módulo / Clase	pantallas.Concurrencia.Exportar → MenuAlumno.exportar("CSV")
Descripción	Exportar la tabla seleccionada a un fichero CSV con separador «:».
Precondiciones	La tabla alumno contiene registros. Pantalla Exportar abierta.
Pasos de ejecución	87. Seleccionar «ALUMNADO». 88. Pulsar el botón «CSV».
Resultado esperado	Se genera el fichero alumno.csv con los campos separados por «:» (método toCSV()). Se muestra «Exportación CSV completada: N registros.».

CP-27 — Exportar la tabla seleccionada a un fichero CSV con separador «:».

Resultado obtenido	Éxito. exportar() escribe alumno.toCSV() por línea. El separador «:» distingue el formato CSV del TXT, conforme a la convención del proyecto.
---------------------------	--

CP-28 · Exportar la tabla seleccionada a un fichero binario .dat mediante serialización de objetos.**CP-28 — Exportar la tabla seleccionada a un fichero binario .dat mediante serialización de objetos.**

Módulo / Clase	pantallas.Concurrencia.Exportar → MenuAlumno.exportar("BINARIO") → GestionFicheros.guardarToBinario
Descripción	Exportar la tabla seleccionada a un fichero binario .dat mediante serialización de objetos.
Precondiciones	La tabla contiene registros. Pantalla Exportar abierta.
Pasos de ejecución	89. Seleccionar la entidad (p. ej. «CICLOS»).
	90. Pulsar el botón «BINARIO».
Resultado esperado	Se genera el fichero .dat con la colección serializada mediante ObjectOutputStream en modo sobrescritura. Para ciclos, la colección se serializa como TreeSet<Ciclo>. Se muestra el total de registros exportados.
Resultado obtenido	Éxito. guardarToBinario() recibe un Collection<?> (polimorfismo) y escribe writeObject(lista). El uso de modo «false» (sobrescritura) evita corromper el binario con cabeceras múltiples.

CP-29 · Exportar la tabla a JSON con un objeto por línea y LocalDate en formato ISO.**CP-29 — Exportar la tabla a JSON con un objeto por línea y LocalDate en formato ISO.**

Módulo / Clase	pantallas.Concurrencia.Exportar → MenuAlumno.exportar("JSON") → Alumno.toJSON / GsonUtils
Descripción	Exportar la tabla a JSON con un objeto por línea y LocalDate en formato ISO.
Precondiciones	La tabla alumno contiene registros con fecha de nacimiento. Pantalla Exportar abierta.
Pasos de ejecución	91. Seleccionar «ALUMNADO».
	92. Pulsar el botón «JSON».

CP-29 — Exportar la tabla a JSON con un objeto por línea y LocalDate en formato ISO.

Resultado esperado	Se genera alumno.json con un objeto JSON por línea; el campo fechaNacimiento aparece como cadena «YYYY-MM-DD» (no como objeto anidado). Se muestra el total de registros exportados.
Resultado obtenido	Éxito. toJSON() usa GsonUtils.get(), cuyo TypeAdapter serializa LocalDate como cadena ISO. Esto garantiza que la reimportación (CP-34) pueda deserializar la fecha correctamente.

CP-30 · Exportar todas las entidades a la vez con la opción «TODO».**CP-30 — Exportar todas las entidades a la vez con la opción «TODO».**

Módulo / Clase	pantallas.Concurrencia.Exportar → ejecutarExportacion("TODO", formato)
Descripción	Exportar todas las entidades a la vez con la opción «TODO».
Precondiciones	Las tablas contienen registros. Pantalla Exportar abierta.
Pasos de ejecución	93. Seleccionar «TODO» en el JComboBox. 94. Pulsar un botón de formato (p. ej. «JSON»).
Resultado esperado	Se exportan secuencialmente alumnado, ciclos, módulos, matrículas y líneas de matrícula al formato elegido, y se muestra «Exportación FORMATO completada: N registros.» con el total acumulado.
Resultado obtenido	Éxito. ejecutarExportacion() recorre todas las entidades llamando a exportarEntidad(t, formato) y suma los registros de cada MenuXxx.exportar(formato).

CP-31 · Importar registros desde un fichero TXT (separador «;») e insertarlos en la BD.**CP-31 — Importar registros desde un fichero TXT (separador «;») e insertarlos en la BD.**

Módulo / Clase	pantallas.Concurrencia.Importar → MenuAlumno.importar("TXT")
Descripción	Importar registros desde un fichero TXT (separador «;») e insertarlos en la BD.
Precondiciones	Existe el fichero alumno.txt con líneas válidas de 6 campos. Pantalla Importar abierta.
Pasos de ejecución	95. Seleccionar «ALUMNADO» en el JComboBox. 96. Pulsar el botón «TXT».

CP-31 — Importar registros desde un fichero TXT (separador «;») e insertarlos en la BD.

Resultado esperado	Se valida que cada línea tenga 6 campos; por cada línea válida se ejecuta un INSERT que ignora el código del fichero (la BD asigna el siguiente). Se muestra «Importados N registros correctamente.».
Resultado obtenido	Éxito. importarAlumnosDesdeTxtCsv() valida el número de campos, normaliza el separador y por cada línea llama a insertarSinID(INSERT_ALUMNO_DESDE_FICHERO[1], datos); el contador solo aumenta cuando la inserción devuelve true.

CP-32 · Importar registros desde un fichero CSV (separador «:»).**CP-32 — Importar registros desde un fichero CSV (separador «:»).**

Módulo / Clase	pantallas.Concurrencia.Importar → MenuAlumno.importar("CSV")
Descripción	Importar registros desde un fichero CSV (separador «:»).
Precondiciones	Existe el fichero alumno.csv con líneas válidas. Pantalla Importar abierta.
Pasos de ejecución	97. Seleccionar «ALUMNADO». 98. Pulsar el botón «CSV».
Resultado esperado	El separador «:» se normaliza internamente a «;» para reutilizar el parseo; se validan 6 campos y se insertan los registros. Se muestra el total importado.
Resultado obtenido	Éxito. Para CSV el método ejecuta linea.replace(":", ";") antes de dividir por «;», reutilizando la misma lógica de importación que el TXT.

CP-33 · Importar registros desde un fichero binario .dat (deserialización de objetos).**CP-33 — Importar registros desde un fichero binario .dat (deserialización de objetos).**

Módulo / Clase	pantallas.Concurrencia.Importar → MenuAlumno.importar("BINARIO")
Descripción	Importar registros desde un fichero binario .dat (deserialización de objetos).
Precondiciones	Existe el fichero .dat generado previamente (CP-28). Pantalla Importar abierta.
Pasos de ejecución	99. Seleccionar la entidad. 100. Pulsar el botón «BINARIO».

CP-33 — Importar registros desde un fichero binario .dat (deserialización de objetos).

Resultado esperado	Se deserializa la colección con <code>ObjectInputStream</code> y se inserta cada elemento; la BD reasigna el código. Se muestra el total importado. Para ciclos, la colección se lee como <code>TreeSet<Ciclo></code> .
Resultado obtenido	Éxito. <code>importarAlumnosDesdeBinario()</code> comprueba primero la existencia del fichero (<code>comprobarFicheroLectura</code>) y recorre la <code>Collection</code> deserializada insertando con <code>insertarSinID</code> .

CP-34 · Importar registros desde un fichero JSON (un objeto por línea) con `LocalDate`.**CP-34 — Importar registros desde un fichero JSON (un objeto por línea) con `LocalDate`.**

Módulo / Clase	<code> pantallas.Concurrencia.Importar → MenuAlumno.importar("JSON") → GsonUtils</code>
Descripción	Importar registros desde un fichero JSON (un objeto por línea) con <code>LocalDate</code> .
Precondiciones	Existe el fichero <code>alumno.json</code> generado en CP-29. Pantalla Importar abierta.
Pasos de ejecución	101. Seleccionar «ALUMNADO». 102. Pulsar el botón «JSON».
Resultado esperado	Cada línea se deserializa a un objeto <code>Alumno</code> ; se valida que el formato sea correcto (nombre no nulo) y se inserta. La fecha ISO se reconstruye como <code>LocalDate</code> . Se muestra el total importado.
Resultado obtenido	Éxito. <code>importarAlumnosDesdeJson()</code> usa <code>GestionFicheros.toJson(linea, Alumno.class)</code> (<code>Gson</code> con <code>TypeAdapter</code> de <code>LocalDate</code>) y descarta líneas inválidas; el contador refleja solo las inserciones correctas.

CP-35 · Importar todas las entidades con la opción «TODO» respetando el orden seguro de claves foráneas.**CP-35 — Importar todas las entidades con la opción «TODO» respetando el orden seguro de claves foráneas.**

Módulo / Clase	<code> pantallas.Concurrencia.Importar → importarTodo(formato)</code>
Descripción	Importar todas las entidades con la opción «TODO» respetando el orden seguro de claves foráneas.

CP-35 — Importar todas las entidades con la opción «TODO» respetando el orden seguro de claves foráneas.

Precondiciones	Existen los ficheros de las distintas entidades. Pantalla Importar abierta. La opción TODO no admite ruta personalizada.
Pasos de ejecución	103. Seleccionar «TODO» en el JComboBox. 104. Pulsar un botón de formato.
Resultado esperado	Se importan las entidades en orden de dependencia: ciclos → módulos → alumnos → matrículas → líneas de matrícula, evitando violaciones de FK. Si un fichero falta o falla, se omite esa tabla y continúa con las demás. Se muestra el total acumulado o «No hay nueva información» si nada se importó.
Resultado obtenido	Éxito. importarTodo() llama en ese orden a MenuCiclo, MenuModulo, MenuAlumno, MenuMatricula y MenuLineaMatricula, sumando los registros; el orden garantiza que las FK existan antes de insertar los dependientes.

CP-36 · Importar desde una ruta de fichero personalizada, detectando el formato por la extensión.**CP-36 — Importar desde una ruta de fichero personalizada, detectando el formato por la extensión.**

Módulo / Clase	pantallas.Concurrencia.Importar → ejecutarImportacion (modo «Otro»)
Descripción	Importar desde una ruta de fichero personalizada, detectando el formato por la extensión.
Precondiciones	Existe un fichero en una ruta concreta (p. ej. /home/usuario/alumno.csv). Pantalla Importar abierta.
Pasos de ejecución	105. Marcar el checkbox «Otro (ruta personalizada)» (habilita el campo de ruta). 106. Seleccionar una entidad concreta (no «TODO».) 107. Escribir la ruta completa con extensión, p. ej. «/home/usuario/alumno.csv». 108. Pulsar cualquier botón de formato.
Resultado esperado	El sistema detecta el formato por la extensión (.csv, .txt, .json, .dat), recorta la extensión y usa la ruta base indicada para importar. Si la ruta está vacía, se selecciona «TODO», o la extensión no se reconoce, se muestra el aviso correspondiente y no se importa.
Resultado obtenido	Éxito. detectarFormatoPorExtension() resuelve el formato; quitarExtension() prepara la ruta base; importarUna() invoca el

CP-36 — Importar desde una ruta de fichero personalizada, detectando el formato por la extensión.

	MenuXxx.importar(formato, rutaBase) adecuado. Los avisos de ruta vacía y «TODO» con ruta personalizada están contemplados.
--	--

CP-37 · Importar desde un fichero vacío o inexistente.**CP-37 — Importar desde un fichero vacío o inexistente.**

Módulo / Clase	pantallas.Concurrencia.Importar → GestionFicheros / Validadores.comprobarFicheroLectura
Descripción	Importar desde un fichero vacío o inexistente.
Precondiciones	El fichero objetivo no existe o tiene tamaño 0 bytes. Pantalla Importar abierta.
Pasos de ejecución	109. Seleccionar una entidad. 110. Pulsar un botón de formato cuyo fichero no exista o esté vacío.
Resultado esperado	No se inserta ningún registro (total = 0) y se muestra el diálogo informativo «No hay nueva información.».
Resultado obtenido	Éxito. comprobarFicheroLectura() detecta que el archivo no existe o está vacío y devuelve false; leerTxtCsv/leerJson devuelven null o lista vacía, por lo que el contador es 0 y la pantalla muestra el mensaje informativo.

3. Prueba Crítica de Excepción (YalImportadoException)

La excepción personalizada YalImportadoException (extends Exception) protege frente a la doble importación de la misma tabla y formato durante una misma sesión. Se demuestra su disparo en el flujo de Consola, donde está implementada mediante banderas estáticas, y se documenta el comportamiento equivalente en la interfaz Swing.

CP-38 · Prueba crítica: al intentar importar por segunda vez una tabla desde el mismo formato en la misma sesión, el sistema lanza YalImportadoException.

CP-38 — Prueba crítica: al intentar importar por segunda vez una tabla desde el mismo formato en la misma sesión, el sistema lanza YalImportadoException.	
Módulo / Clase	Menus.MenuAlumno.mostrarMenuImportar → importarDesdeTxt (excepciones.YalImportadoException)
Descripción	Prueba crítica: al intentar importar por segunda vez una tabla desde el mismo formato en la misma sesión, el sistema lanza YalImportadoException.
Precondiciones	Aplicación arrancada en modo Consola. La tabla alumno YA ha sido importada desde TXT una vez en esta sesión (la bandera estática importadoTxt = true). Existe alumno.txt.
Pasos de ejecución	111. Entrar en GESTIÓN DE ALUMNOS → opción 7 (Importar tabla). 112. Elegir opción 1 (Importar desde TXT) por primera vez → importación correcta (importadoTxt pasa a true). 113. Volver a entrar en Importar tabla y elegir de nuevo opción 1 (Importar desde TXT).
Resultado esperado	En el segundo intento, el método importarDesdeTxt() detecta que importadoTxt es true y lanza «throw new YalImportadoException("La tabla alumno ya fue importada desde TXT en esta sesión.")». La excepción es capturada en mostrarMenuImportar y se muestra «[AVISO] La tabla alumno ya fue importada desde TXT en esta sesión.». No se duplican datos.
Resultado obtenido	Éxito. El primer import ejecuta correctamente y activa la bandera. El segundo dispara la YalImportadoException, capturada por el bloque «catch (YalImportadoException e)» que imprime el mensaje precedido de «[AVISO]». El control vuelve al menú sin insertar registros, demostrando la protección contra la doble carga. El mismo patrón está implementado para CSV (importadoCsv), Binario (importadoBin) y JSON (importadoJson), y de forma análoga en MenuCiclo, MenuModulo, MenuMatricula y MenuLineaMatricula.

CP-39 · Comportamiento equivalente en la interfaz Swing al reintentar la importación de una tabla ya cargada.

CP-39 — Comportamiento equivalente en la interfaz Swing al reintentar la importación de una tabla ya cargada.	
Módulo / Clase	pantallas.Concurrencia.Importar → MenuAlumno.importar + GestionBaseDeDatos.insertarSinID
Descripción	Comportamiento equivalente en la interfaz Swing al reintentar la importación de una tabla ya cargada.
Precondiciones	Pantalla Swing Importar abierta. La tabla alumno ya fue importada una vez (sus correos, que son UNIQUE, ya existen en la BD).
Pasos de ejecución	114. Seleccionar «ALUMNADO». 115. Pulsar «TXT» para importar. 116. Pulsar «TXT» una segunda vez con el mismo fichero.
Resultado esperado	En el segundo intento, los INSERT fallan por violación de la restricción UNIQUE del correo; insertarSinID devuelve false para cada registro, el contador permanece a 0 y la pantalla muestra «No hay nueva información.». No se crean registros duplicados.
Resultado obtenido	Éxito. A diferencia del flujo de Consola (que usa YaImportadoException), la pantalla Swing delega en MenuAlumno.importar(formato), cuyas inserciones duplicadas son rechazadas por la BD; insertarSinID captura la SQLException y devuelve false, de modo que total = 0 y se informa «No hay nueva información.». El resultado funcional —evitar la duplicación— es el mismo.

4. Pruebas de Interfaz (Validaciones y Navegación)

Se comprueba cómo el sistema gestiona datos incorrectos, campos vacíos y la navegación, tanto en la interfaz Swing (validaciones campo a campo con diálogos JOptionPane y la clase Validadores) como en la Consola (control de InputMismatchException, DateTimeParseException y opciones de menú no válidas).

CP-40 · Validación Swing: intentar guardar un alumno con el campo Nombre vacío.

CP-40 — Validación Swing: intentar guardar un alumno con el campo Nombre vacío.	
Módulo / Clase	pantallas.Crear.CrearAlumno.crearAlumno → Validadores.validarTextoNoVacio
Descripción	Validación Swing: intentar guardar un alumno con el campo Nombre vacío.
Precondiciones	Pantalla CrearAlumno abierta.
Pasos de ejecución	117. Dejar el campo Nombre vacío. 118. Rellenar el resto de campos con valores válidos. 119. Pulsar «Siguiente».
Resultado esperado	Aparece el diálogo «El nombre no puede estar vacío.» y no se realiza ninguna inserción en la BD.
Resultado obtenido	Éxito. crearAlumno() evalúa primero !Validadores.validarTextoNoVacio(nombre); al ser cierto, muestra el JOptionPane y ejecuta return antes de llegar al INSERT.

CP-41 · Validación Swing: correo con formato inválido.

CP-41 — Validación Swing: correo con formato inválido.	
Módulo / Clase	pantallas.Crear.CrearAlumno.crearAlumno → Validadores.validarCorreo
Descripción	Validación Swing: correo con formato inválido.
Precondiciones	Pantalla CrearAlumno abierta.
Pasos de ejecución	120. Rellenar el Nombre con un valor válido. 121. Introducir Correo: «laura.gomez(sin arroba)». 122. Pulsar «Siguiente».
Resultado esperado	Aparece el diálogo «El correo no tiene un formato válido (usuario@dominio.ext).» y no se inserta nada.

CP-41 — Validación Swing: correo con formato inválido.

Resultado obtenido	Éxito. validarCorreo() aplica la expresión regular <code>^[w.-]+@[w.-]+\.[a-zA-Z]{2,}\$</code> ; al no cumplirse, se muestra el aviso y se aborta con return.
---------------------------	--

CP-42 · Validación Swing: teléfono que no tiene exactamente 9 dígitos.**CP-42 — Validación Swing: teléfono que no tiene exactamente 9 dígitos.**

Módulo / Clase	pantallas.Crear.CrearAlumno.crearAlumno → Validadores.validarTelefono
Descripción	Validación Swing: teléfono que no tiene exactamente 9 dígitos.
Precondiciones	Pantalla CrearAlumno abierta, demás campos válidos.
Pasos de ejecución	123. Introducir Teléfono: «12345» (menos de 9 dígitos) o «123ABC789» (con letras). 124. Pulsar «Siguiente».
Resultado esperado	Aparece el diálogo «El teléfono debe tener exactamente 9 dígitos numéricos.» y no se inserta nada.
Resultado obtenido	Éxito. validarTelefono() comprueba <code>telefono.matches("\\d{9}")</code> ; cualquier valor que no sean 9 dígitos numéricos provoca el aviso y el return.

CP-43 · Validación Swing: domicilio que no respeta el formato exigido.**CP-43 — Validación Swing: domicilio que no respeta el formato exigido.**

Módulo / Clase	pantallas.Crear.CrearAlumno.crearAlumno → Validadores.validarDireccion
Descripción	Validación Swing: domicilio que no respeta el formato exigido.
Precondiciones	Pantalla CrearAlumno abierta, demás campos válidos.
Pasos de ejecución	125. Introducir Domicilio: «Madrid» (sin tipo de vía, número, localidad ni provincia). 126. Pulsar «Siguiente».
Resultado esperado	Aparece el diálogo de domicilio inválido (debe seguir el formato «TipoVía NombreVía Número, Localidad, Provincia», p. ej. «Calle Mayor 5, Madrid, Madrid») y no se inserta nada.
Resultado obtenido	Éxito. validarDireccion() exige al menos 3 partes separadas por comas, que la primera tenga ≥ 3 tokens y que el último sea un número (patrón <code>\d+[A-Za-z]*</code>), y que localidad y provincia no estén vacías. «Madrid» no cumple, por lo que se aborta.

CP-44 · Validación Swing: fecha de nacimiento vacía, con formato incorrecto o en el futuro.

CP-44 — Validación Swing: fecha de nacimiento vacía, con formato incorrecto o en el futuro.	
Módulo / Clase	pantallas.Crear.CrearAlumno.crearAlumno → Validadores.validarFechaNacimiento
Descripción	Validación Swing: fecha de nacimiento vacía, con formato incorrecto o en el futuro.
Precondiciones	Pantalla CrearAlumno abierta, demás campos válidos.
Pasos de ejecución	127. Introducir una fecha futura como «2999-01-01» o con formato erróneo como «13/03/2002». 128. Pulsar «Siguiente».
Resultado esperado	Para fecha vacía: «La fecha de nacimiento no puede estar vacía.». Para fecha futura o formato no ISO: «La fecha debe ser YY-MM-DD con guiones». En ningún caso se inserta el alumno.
Resultado obtenido	Éxito. Se comprueba primero validarTextoNoVacio sobre la fecha y después validarFechaNacimiento, que intenta LocalDate.parse(temp) y rechaza fechas nulas o posteriores a LocalDate.now(). Ambos caminos muestran el diálogo y ejecutan return.

CP-45 · Validación Swing: intentar eliminar sin seleccionar fila o sin introducir teléfono y correo.

CP-45 — Validación Swing: intentar eliminar sin seleccionar fila o sin introducir teléfono y correo.	
Módulo / Clase	pantallas.Eliminar.EliminarAlumno.eliminarAlumnoSeleccionado
Descripción	Validación Swing: intentar eliminar sin seleccionar fila o sin introducir teléfono y correo.
Precondiciones	Pantalla EliminarAlumno abierta con la tabla cargada.
Pasos de ejecución	129. No seleccionar ninguna fila de la tabla y pulsar eliminar (caso A). 130. Seleccionar una fila pero dejar vacíos teléfono y/o correo y pulsar eliminar (caso B).
Resultado esperado	Caso A: «Debes seleccionar el alumno de la tabla.». Caso B: «Debes introducir teléfono y correo.». En ambos casos no se ejecuta el borrado.

CP-45 — Validación Swing: intentar eliminar sin seleccionar fila o sin introducir teléfono y correo.

Resultado obtenido	Éxito. eliminarAlumnoSeleccionado() verifica primero la selección de la tabla y después que teléfono y correo no estén vacíos; cada comprobación fallida muestra su JOptionPane y aborta con return antes de llamar a eliminarAlumnoCompletoPorCodigoTelefonoCorreo().
---------------------------	---

CP-46 · Navegación Consola: introducir una opción de menú fuera del rango permitido.

CP-46 — Navegación Consola: introducir una opción de menú fuera del rango permitido.

Módulo / Clase	Menus.MenuAlumno.mostrarMenu (navegación de Consola)
Descripción	Navegación Consola: introducir una opción de menú fuera del rango permitido.
Precondiciones	Aplicación en modo Consola, dentro del menú GESTIÓN DE ALUMNOS.
Pasos de ejecución	131. Introducir «15» cuando se pide «Elija una opción (1-9)».
Resultado esperado	Se muestra «[ERROR] Opción no válida. Introduzca un número entre 1 y 9.» y el menú se vuelve a mostrar sin salir.
Resultado obtenido	Éxito. El switch sobre la opción cae en la rama default, que imprime el mensaje de error; el bucle while (!volver) repite el menú.

CP-47 · Validación Consola: introducir texto donde se espera un número entero (opción de menú o código).

CP-47 — Validación Consola: introducir texto donde se espera un número entero (opción de menú o código).

Módulo / Clase	Menus.MenuAlumno.mostrarMenu → InputMismatchException
Descripción	Validación Consola: introducir texto donde se espera un número entero (opción de menú o código).
Precondiciones	Aplicación en modo Consola, en un punto donde se lee un entero con teclado.nextInt().
Pasos de ejecución	132. Introducir «abc» cuando se solicita un número.
Resultado esperado	Se captura InputMismatchException y se muestra «[ERROR] Debe introducir un número entero.»; se limpia el búfer del Scanner (teclado.nextLine()) para no entrar en bucle infinito.

CP-47 — Validación Consola: introducir texto donde se espera un número entero (opción de menú o código).

Resultado obtenido	Éxito. El bloque «catch (InputMismatchException e)» imprime el mensaje y ejecuta teclado.nextLine() para descartar la entrada inválida, permitiendo continuar.
---------------------------	---

CP-48 · Validación Consola: fecha de nacimiento con formato distinto de dd/MM/yyyy al insertar un alumno.**CP-48 — Validación Consola: fecha de nacimiento con formato distinto de dd/MM/yyyy al insertar un alumno.**

Módulo / Clase	Menus.MenuAlumno.insertarAlumno → DateTimeParseException
Descripción	Validación Consola: fecha de nacimiento con formato distinto de dd/MM/yyyy al insertar un alumno.
Precondiciones	Aplicación en modo Consola, opción «Insertar alumno».
Pasos de ejecución	133. Introducir el nombre. 134. Introducir la fecha como «2002-03-13» (formato ISO en lugar de dd/MM/yyyy).
Resultado esperado	LocalDate.parse(fechaStr, FORMATO_FECHA) lanza DateTimeParseException, capturada, y se muestra «[ERROR] Formato de fecha incorrecto. Use dd/MM/yyyy.». No se inserta el alumno.
Resultado obtenido	Éxito. El FORMATO_FECHA del menú de consola es dd/MM/yyyy; una entrada en otro formato dispara la excepción, que el catch traduce a un mensaje claro para el usuario.

CP-49 · Validación Consola: insertar un alumno con teléfono o correo inválidos.**CP-49 — Validación Consola: insertar un alumno con teléfono o correo inválidos.**

Módulo / Clase	Menus.MenuAlumno.insertarAlumno → Validadores (teléfono/correo)
Descripción	Validación Consola: insertar un alumno con teléfono o correo inválidos.
Precondiciones	Aplicación en modo Consola, opción «Insertar alumno», con fecha en formato correcto.
Pasos de ejecución	135. Introducir un teléfono de menos de 9 dígitos o un correo sin «@».
Resultado esperado	Se muestra «[ERROR] Datos inválidos. Compruebe el teléfono (9 dígitos) y el correo.» y no se ejecuta el INSERT.

CP-49 — Validación Consola: insertar un alumno con teléfono o correo inválidos.**Resultado obtenido**

Éxito. insertarAlumno() combina las comprobaciones validarTextoNoVacio, validarTelefono y validarCorreo en una condición; si alguna falla, imprime el error y aborta con return antes de insertarDatos().

5. Informes Multitabla (Consultas de Ejercicios)

Se verifican los nueve informes multitabla definidos en ConsultasEjercicios y ejecutados desde la pantalla Swing Ejercicios. La pantalla obtiene la SQL del ejercicio seleccionado, solicita por diálogo los parámetros necesarios (ejercicios 2, 3 y 4) y vuelca el resultado en una jTable mediante obtenerTableModel. Todas las consultas combinan varias tablas con JOIN y aplican el orden indicado en el enunciado.

CP-50 · Informe 1: denominación, familia profesional y nivel de todos los ciclos con los nombres y horas de sus módulos, ordenado por denominación del ciclo (ASC).

CP-50 — Informe 1: denominación, familia profesional y nivel de todos los ciclos con los nombres y horas de sus módulos, ordenado por denominación del ciclo (ASC).	
Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta1
Descripción	Informe 1: denominación, familia profesional y nivel de todos los ciclos con los nombres y horas de sus módulos, ordenado por denominación del ciclo (ASC).
Precondiciones	Existen ciclos con módulos asociados. Pantalla Ejercicios abierta.
Pasos de ejecución	136. Seleccionar «Ejercicio 1» en el desplegable. 137. Pulsar «Actualizar».
Resultado esperado	La jTable muestra una fila por cada módulo con su ciclo (JOIN ciclo–módulo), ordenada por c.denominacion ASC. No se solicitan parámetros.
Resultado obtenido	Éxito. obtenerParametros(1) devuelve un array vacío y obtenerTableModel ejecuta el JOIN entre ciclo y modulo con ORDER BY c.denominacion ASC.

CP-51 · Informe 2: nombre y horas de los módulos de un nivel de ciclo y un curso dados, con la denominación del ciclo y el número de alumnos matriculados, ordenado por nombre del módulo.

CP-51 — Informe 2: nombre y horas de los módulos de un nivel de ciclo y un curso dados, con la denominación del ciclo y el número de alumnos matriculados, ordenado por nombre del módulo.	
Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta2 (parámetros)
Descripción	Informe 2: nombre y horas de los módulos de un nivel de ciclo y un curso dados, con la denominación del ciclo y el número de alumnos matriculados, ordenado por nombre del módulo.

CP-51 — Informe 2: nombre y horas de los módulos de un nivel de ciclo y un curso dados, con la denominación del ciclo y el número de alumnos matriculados, ordenado por nombre del módulo.

Precondiciones	Existen datos relacionados de ciclo, módulo, línea de matrícula, matrícula y alumno. Pantalla Ejercicios abierta.
Pasos de ejecución	138. Seleccionar «Ejercicio 2». 139. Pulsar «Actualizar». 140. Introducir el Nivel solicitado por el diálogo (p. ej. «Superior»). 141. Introducir el Curso solicitado (p. ej. «primero»).
Resultado esperado	La consulta filtra por c.nivel y m.curso, agrupa y cuenta los alumnos matriculados (COUNT) y ordena el resultado. La jTable muestra las filas resultantes.
Resultado obtenido	Éxito. pedirParametrosEjercicio2() solicita nivel y curso mediante JOptionPane.showInputDialog y los pasa como parámetros del PreparedStatement (WHERE c.nivel = ? AND m.curso = ?).

CP-52 · Informe 3: nombre y fecha de nacimiento de los alumnos y los módulos en que están matriculados en un año académico dado, ordenado por nombre del alumno.
CP-52 — Informe 3: nombre y fecha de nacimiento de los alumnos y los módulos en que están matriculados en un año académico dado, ordenado por nombre del alumno.

Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta3 (parámetro)
Descripción	Informe 3: nombre y fecha de nacimiento de los alumnos y los módulos en que están matriculados en un año académico dado, ordenado por nombre del alumno.
Precondiciones	Existen matrículas del año consultado. Pantalla Ejercicios abierta.
Pasos de ejecución	142. Seleccionar «Ejercicio 3». 143. Pulsar «Actualizar». 144. Introducir el Año académico solicitado (p. ej. «2024»).
Resultado esperado	La consulta filtra por ma.anio_academico = ? y ordena por a.nombre ASC. La jTable muestra alumno, fecha de nacimiento y nombre del módulo.
Resultado obtenido	Éxito. pedirParametrosEjercicio3() solicita el año y lo pasa como único parámetro; el JOIN entre módulo, ciclo, línea de matrícula, matrícula y alumno produce el informe.

CP-53 · Informe 4: nombre y fecha de nacimiento de los alumnos matriculados en un curso de un ciclo y año concretos, con sus calificaciones primera y segunda, ordenado por nombre del alumno.

CP-53 — Informe 4: nombre y fecha de nacimiento de los alumnos matriculados en un curso de un ciclo y año concretos, con sus calificaciones primera y segunda, ordenado por nombre del alumno.

Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta4 (parámetros)
Descripción	Informe 4: nombre y fecha de nacimiento de los alumnos matriculados en un curso de un ciclo y año concretos, con sus calificaciones primera y segunda, ordenado por nombre del alumno.
Precondiciones	Existen líneas de matrícula con calificaciones para la combinación consultada. Pantalla Ejercicios abierta.
Pasos de ejecución	145. Seleccionar «Ejercicio 4». 146. Pulsar «Actualizar». 147. Introducir la Denominación del ciclo. 148. Introducir el Curso. 149. Introducir el Año académico.
Resultado esperado	La consulta filtra por c.denominacion = ?, m.curso = ? y ma.anio_academico = ? y muestra las calificaciones, ordenado por a.nombre ASC.
Resultado obtenido	Éxito. pedirParametrosEjercicio4() solicita los tres valores en orden; si el usuario cancela cualquiera, devuelve null y no se ejecuta la consulta (ver CP-58).

CP-54 · Informe 5: denominación, familia profesional y nivel de los ciclos con el importe total de las matrículas por año académico, ordenado por denominación.

CP-54 — Informe 5: denominación, familia profesional y nivel de los ciclos con el importe total de las matrículas por año académico, ordenado por denominación.

Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta5
Descripción	Informe 5: denominación, familia profesional y nivel de los ciclos con el importe total de las matrículas por año académico, ordenado por denominación.
Precondiciones	Existen matrículas con importe. Pantalla Ejercicios abierta.

CP-54 — Informe 5: denominación, familia profesional y nivel de los ciclos con el importe total de las matrículas por año académico, ordenado por denominación.

Pasos de ejecución	150. Seleccionar «Ejercicio 5». 151. Pulsar «Actualizar».
Resultado esperado	La jTable muestra el importe total agrupado por ciclo, familia, nivel y año (SUM(ma.importe)), ordenado por denominación. No requiere parámetros.
Resultado obtenido	Éxito. La consulta agrupa con GROUP BY y agrega con SUM; obtenerParametros(5) devuelve un array vacío.

CP-55 · Informe 6: nombre del alumno y año académico con el total de créditos ECTS y total de horas matriculadas, ordenado por total de créditos (DESC).
CP-55 — Informe 6: nombre del alumno y año académico con el total de créditos ECTS y total de horas matriculadas, ordenado por total de créditos (DESC).

Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta6
Descripción	Informe 6: nombre del alumno y año académico con el total de créditos ECTS y total de horas matriculadas, ordenado por total de créditos (DESC).
Precondiciones	Existen matrículas con módulos. Pantalla Ejercicios abierta.
Pasos de ejecución	152. Seleccionar «Ejercicio 6». 153. Pulsar «Actualizar».
Resultado esperado	La jTable muestra los totales SUM(creditos_ects) y SUM(horas) por alumno y año, ordenados de forma descendente por créditos.
Resultado obtenido	Éxito. La consulta usa GROUP BY a.nombre, ma.anio_academico y ORDER BY SUM(m.creditos_ects) DESC, mostrando primero los alumnos con más créditos.

CP-56 · Informe 7: nombre del alumno, nombre del módulo y año académico de los registros cuya calificación primera aún es nula, ordenado por año académico.
CP-56 — Informe 7: nombre del alumno, nombre del módulo y año académico de los registros cuya calificación primera aún es nula, ordenado por año académico.

Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta7
-----------------------	---

CP-56 — Informe 7: nombre del alumno, nombre del módulo y año académico de los registros cuya calificación primera aún es nula, ordenado por año académico.

Descripción	Informe 7: nombre del alumno, nombre del módulo y año académico de los registros cuya calificación primera aún es nula, ordenado por año académico.
Precondiciones	Existen líneas de matrícula con calificacion_primera a NULL. Pantalla Ejercicios abierta.
Pasos de ejecución	154. Seleccionar «Ejercicio 7». 155. Pulsar «Actualizar».
Resultado esperado	La jTable muestra solo los registros con lm.calificacion_primera IS NULL, ordenados por ma.anio_academico ASC.
Resultado obtenido	Éxito. El filtro WHERE lm.calificacion_primera IS NULL aísla las calificaciones pendientes y el ORDER BY las ordena por año.

CP-57 · Informe 8: nombre del módulo y denominación del ciclo, indicando cuántos alumnos están matriculados con repetición superior a 1, mostrando solo módulos con más de 3 repetidores.

CP-57 — Informe 8: nombre del módulo y denominación del ciclo, indicando cuántos alumnos están matriculados con repetición superior a 1, mostrando solo módulos con más de 3 repetidores.

Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta8
Descripción	Informe 8: nombre del módulo y denominación del ciclo, indicando cuántos alumnos están matriculados con repetición superior a 1, mostrando solo módulos con más de 3 repetidores.
Precondiciones	Existen líneas de matrícula con repeticion > 1. Pantalla Ejercicios abierta.
Pasos de ejecución	156. Seleccionar «Ejercicio 8». 157. Pulsar «Actualizar».
Resultado esperado	La jTable muestra los módulos con COUNT de repetidores filtrado por lm.repeticion > 1 y HAVING COUNT(...) > 3.
Resultado obtenido	Éxito. La consulta combina WHERE lm.repeticion > 1 con GROUP BY y HAVING COUNT(lm.codigo_matricula) > 3, devolviendo únicamente los módulos con más de 3 repetidores.

CP-58 · Informe 9: nombre, teléfono y correo de los alumnos sin ninguna matrícula o con matrícula en estado «Anulada», ordenado por nombre.

CP-58 — Informe 9: nombre, teléfono y correo de los alumnos sin ninguna matrícula o con matrícula en estado «Anulada», ordenado por nombre.	
Módulo / Clase	pantallas.Consultas.Ejercicios → ConsultasEjercicios.datosConsulta9
Descripción	Informe 9: nombre, teléfono y correo de los alumnos sin ninguna matrícula o con matrícula en estado «Anulada», ordenado por nombre.
Precondiciones	Existen alumnos sin matrícula y/o con matrícula anulada. Pantalla Ejercicios abierta.
Pasos de ejecución	158. Seleccionar «Ejercicio 9». 159. Pulsar «Actualizar».
Resultado esperado	La jTable muestra los alumnos cuyo LEFT JOIN con matricula es NULL o cuyo estado es «Anulada», sin duplicados (DISTINCT) y ordenados por a.nombre ASC.
Resultado obtenido	Éxito. El LEFT JOIN con la condición WHERE mat.codigo IS NULL OR mat.estado = 'Anulada' recupera ambos colectivos de alumnos en un solo informe.

CP-59 · Informe parametrizado: cancelar el diálogo de introducción de parámetros.

CP-59 — Informe parametrizado: cancelar el diálogo de introducción de parámetros.	
Módulo / Clase	pantallas.Consultas.Ejercicios.cargarTabla (control de cancelación)
Descripción	Informe parametrizado: cancelar el diálogo de introducción de parámetros.
Precondiciones	Pantalla Ejercicios abierta con un ejercicio que requiere parámetros (2, 3 o 4) seleccionado.
Pasos de ejecución	160. Seleccionar «Ejercicio 4». 161. Pulsar «Actualizar». 162. Pulsar «Cancelar» en cualquiera de los diálogos de parámetros.
Resultado esperado	La consulta no se ejecuta y la tabla permanece sin cambios; no se produce ningún error.
Resultado obtenido	Éxito. Cuando el usuario cancela, el método pedirParametrosEjercicioN() devuelve null; cargarTabla() comprueba «if (parametros == null) return;» y aborta sin lanzar la consulta, evitando errores por parámetros incompletos.

